Assignment 3 - Constructors, destructors, and operator overloading

Course outcome: CO3 Student: Hitesh Register number: 192524005 Compiler: LLVM-MinGW `g++` driver, Clang 23.1.1, `x86_64-w64-windows-gnu` Language standard: C++17 Report date: 7 November 2026 Repository: `https://github.com/Hitesh-92007/DSA0101-192524005`

Abstract

This assignment implements and evaluates C++ constructors, copy behavior, destructors, dynamic allocation, and operator overloading. Seven programs model library book records, hospital prescriptions, auditorium seating, rupee arithmetic, a music playlist timer, owned text, and complex circuit impedance. The work includes compiler diagnostics, expected output reasoning, sanitizer runs, memory diagrams, and timing measurements. The measurements and outputs are stored beside the source files in the task folders.

Build and evaluation method

The normal build used C++17 and `-Wall -Wextra -Wconversion -pedantic -O2`. The compiler was the LLVM-MinGW `g++` driver reporting Clang 23.1.1 for a Windows GNU target. Sanitizer variants used `-fsanitize=address -fno-omit-frame-pointer -O1 -g`. Q5 was compiled with both `-O0` and `-O2`; Q4 and Q6 were benchmarked in five separate process runs. Raw results are in `../BENCHMARK_RUNS.txt` and `../BUILD_RESULTS.md`.

The task-level prediction notes were prepared by the assistant before the final verification rerun documented below. Earlier exploratory executions had already occurred, so these notes are not a claim that predictions were written before the first execution or independently by Hitesh. Text run transcripts and source/output PNG views rendered from those files are included. The PNGs are labeled views, not direct screen captures. A genuine Notepad capture of Task 1 output was displayed in the conversation but could not be saved into the package.

Q1 - Constructors and constructor overloading

Design

`Book` has three member fields-title, author, and number of copies-and represents three library situations: an empty record, a title-only donated book, and a fully catalogued book. The default and one-argument constructors delegate to the full constructor. The full constructor initializes members through its initializer list and rejects negative copy counts.

The compiler-supplied default constructor exists only when no constructor is declared by the class. Because this `Book` declares constructors, its default constructor is explicitly supplied. `q1_no_default_array.cpp` removes it; the compiler then rejects `Book books[2]`. `q1_explicit_probe.cpp` compiles a copy-initialization from integer to `PageCount` when the constructor is implicit and fails when `explicit` is enabled.

Members initialize in declaration order regardless of the order written in the initializer list. The deliberate `ReorderProbe` emits `-Wreorder-ctor`, while its values still print as `1 2`. Initializer lists initialize class members directly; assignments in the constructor body would first default-construct class-type members and then assign them. This is also the only way to initialize `const` and reference members.

Results and complexity

The output confirms all three states, creates a vector of two default records, and rejects `-1` copies. The `Book` state uses O(1) scalar fields plus string storage proportional to title and author length. Construction is O(1) for scalars and O(length) when strings are copied. A constructor with default arguments would have the same asymptotic cost and can reduce overload count, but explicit overloads make these three domain states easier to distinguish and let delegation centralize validation.

The constructor from one argument is marked explicit when implicit conversion would be surprising or unsafe. For example, `PageCount pages = 300` compiles without `explicit` and fails with it. See `Task01/answer.md` and the compile diagnostics.

Q2 - Copy constructors: shallow and deep copies

Design and ownership

`Prescription` owns an array of `std::string` values. The shallow variant copies the raw pointer, so both prescriptions refer to the same allocation. The deep variant allocates a separate array and copies each medicine. Its copy assignment checks self-assignment, builds the replacement first, then deletes the old array. Together with the destructor, the copy constructor and copy assignment implement the Rule of Three.

```
Shallow copy (unsafe)
p ── medicines_ ──┐
                  ├──> [Paracetamol] [ORS]
q ── medicines_ ──┘        same array; two destructors try to delete it

Deep copy (safe)
p ── medicines_ ─────> [Paracetamol] [ORS]
q ── medicines_ ─────> [Paracetamol] [ORS]
                         separate arrays; changing q leaves p unchanged
```

Results, complexity, and alternatives

The output matches the supplied example: `Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS`. This demonstrates that changing `p` does not change `q` or `r`. Self-assignment preserves `p`; pass-by-value makes one copy, while pass-by-const-reference makes none. AddressSanitizer reports a double free for the shallow version and exits cleanly for the deep version.

Copying `n` medicine names takes O(n + total characters) time and O(n + total characters) additional storage. A shallow pointer copy is O(1), but it violates the independent ownership contract. Replacing the pointer with `std::vector<std::string>` gives a deep copy through the container and follows the Rule of Zero; the same O(n) copy cost remains, while lifetime and exceptions are managed by the library.

The program measures repeated copies of 10, 1,000, and 100,000 medicine entries. The exact totals and repetition counts are in `Task02/RUN_OUTPUT.txt`; these are microbenchmarks and are affected by allocation and machine noise. See `Task02/answer.md`.

Q3 - Dynamic constructors and destructors

Design and memory layout

`Event` constructs a row-pointer table and then one character array per row. It initializes free seats to `.` and changes booked seats to `X`. The constructor releases any partially allocated rows if construction throws. The destructor releases each row with `delete[]`, then the row-pointer array. Copying is disabled to avoid accidental pointer aliasing. The heap-created event is explicitly deleted; a macro build deliberately omits that `delete`.

```
Event object
  name, rows, seats-per-row, booked-count
  seats_ ──> row-pointer table [row0*, row1*, ...]
                row0* ──> ['.', 'X', '.', ...]
                row1* ──> ['.', '.', '.', ...]
```

...

On a 64-bit system, the array payload estimate is `rows x sizeof(char*) + rows x seats x sizeof(char)`: 280 B for 10x20, 20,800 B for 100x200, and 2,008,000 B for 1,000x2,000. This excludes allocator headers, object fields, and the event name. All maps are O(rowsxseats) space; booking and cancellation are O(1).

Lifetime and experiment

The stack event is destroyed at the end of `main`; the block event is destroyed at the closing brace; the heap event is destroyed at its `delete`. This order appears in `Task03/RUN_OUTPUT.txt`. Omitting `delete` leaks the object, its row-pointer table, and the row arrays. Using scalar `delete` for an array allocated by `new[]` would be undefined behavior. A destructor has no arguments and cannot be overloaded; it releases the object's owned resources.

AddressSanitizer reports no address error for the correct program. The missing-delete variant runs, but the Windows AddressSanitizer runtime says LeakSanitizer is unsupported, so it cannot report the leak. A Linux WSL distribution was not installed. This environment limit is documented rather than treated as a clean leak check. A contiguous `rows x seats` array would use one allocation and no row-pointer table but would require equal row lengths; `std::vector<std::vector<char>>` supports varying rows and manages memory automatically.

Q4 - Arithmetic and stream operators for money

Design and implementation

`Money` stores total paise in a `long long`, avoiding binary floating-point rounding in currency arithmetic. The class implements `+`, `-`, `+=`, `==`, and `<`; stream extraction parses decimal digits, and insertion prints rupees with two paise digits. `+` returns a new value, while `+=` updates the left operand and returns a reference to it.

Arithmetic operators are members in this design. A non-member `operator+` could allow implicit conversion on either side (`money + 100` and `100 + money`); the current member operator supports the money-left form. Stream operators are non-members because `std::cout` or `std::cin` is the left operand. The compiler translates member `a + b` as `a.operator+(b)`. Multiplying two money values usually has no useful currency unit, so a tax or discount should be modeled as a scalar rate with a stated rounding policy.

Results and measurement

The output is `Rs 200.00` for Rs 120.75 + Rs 79.25, reports equality as true, carries Rs 0.75 + Rs 0.50 to Rs 1.25, parses Rs 12.34, and represents subtraction as Rs -0.75. In five runs of one million additions, median integer-paise time was 1,001 us (range 693-1,337 us); median double time was 3,219 us (range 2,584-4,829 us). The repeated double increment ended at `10000.000000171856` instead of exactly `10000`, while integer arithmetic remained exact. Both approaches are O(1) per operation and O(1) additional space. The timings are specific to this build and system.

Q5 - Increment, comparison, and subscript operators

Design and reference prediction

`Time` stores seconds since midnight. Prefix `++` advances and returns a reference; postfix `++(int)` saves the old value and returns that copy. Both wrap after 23:59:59. `==` and `<` compare the stored seconds. `Playlist` stores tracks in a vector and offers const and mutable `operator[]` overloads. Both check the range and throw `std::out_of_range` for invalid indices.

For the supplied `Counter` example, `a = c++` gives `a=5` and leaves `c=6`; `b = ++c` gives `b=7` and leaves `c=7`. Therefore the expected output is `5 7 7`. The run confirms this. It also verifies `00:00:59 -> 00:01:00`, `00:59:59 -> 01:00:00`, and `23:59:59 -> 00:00:00`; both `playlist[-1]` and `playlist[size]` are rejected.

Complexity and experiment

Increment, comparison, and playlist indexing take O(1) time and O(1) extra space. Postfix returns the old state and therefore creates a copy for this class; prefix returns a reference. Five runs of 10 million increments had medians of 56,888 us prefix and 58,627 us postfix at `-O0`; at `-O2` the medians were 54,184 us and 52,531 us. Results overlapped and varied substantially, so they do not show a stable timing advantage for this small object on this machine. Prefix avoids the copy by definition when the old value is not needed. See `BENCHMARK_RUNS.txt` for all measurements.

Q6 - The rules of operator overloading and an owned Text class

Implementation and rule answers

`Text` owns a dynamically allocated null-terminated character array and implements construction, deep copy construction, safe copy assignment, destruction, concatenation `+`, equality, inequality, and stream insertion. Assignment allocates the replacement before releasing the old buffer and handles self-assignment. Empty and long strings are tested.

The overloadable syntax cannot change operator precedence, associativity, or arity, and at least one operand must be a user-defined type. Operators that cannot be overloaded include `::`, `.`, `.*`, `?:`, and `sizeof`. Assignment must be a member. A non-member is appropriate when the left operand is not `Text` (for example, `std::ostream << text`) or when symmetric conversions are needed. Friendship should be limited to non-members that need private access.

In `a+b+c`, left associativity forms `(a+b)+c`. Under C++17, returned prvalues are constructed directly in this expression; the measured explicit copy-constructor count is zero. The concatenation still allocates result buffers and copies characters. A single concatenation of lengths n1 and n2 is O(n1+n2); `a+b+c` takes O(n1+n2+n3), with an intermediate result. Repeated `result = result + part` can copy quadratically many characters, so reserve-and-append or `std::string` is more suitable for repeated building.

Results and experiment

AddressSanitizer reports no memory error. The five-run median for 100,000 short concatenations was 5,086 us for `Text` and 3,607 us for `std::string`. The checksums match; `std::string` is faster in the median local run and handles capacity and lifetime concerns more robustly. The custom class is an educational example; production code should normally use `std::string` and follow the Rule of Zero.

Q7 - Complex number class and RLC impedance

Design and formulas

`Complex` stores two doubles and provides default, real-only, and real-plus-imaginary constructors. It overloads addition, subtraction, multiplication, division, equality, and rectangular output; `magnitude()` uses `std::hypot`. Division checks for an exactly zero denominator and throws `std::domain_error`.

For `a=x+jy` and `b=u+jv`, multiplication is `(xu-yv)+j(xv+yu)` and division is `((xu+yv)+j(yu-xv))/(u²+v²)`. Each operation uses O(1) time and O(1) extra space. A parallel pair is `(Z1xZ2)/(Z1+Z2)`.

Circuit calculation and verification

The example uses R=10 ohm, L=0.1 H, C=100 uF, and f=50 Hz. Then `ω=2πf≈314.1593 rad/s`, `ZL=jωL≈j31.4159 ohm`, and `ZC=-j/(ωC)≈-j31.8310 ohm`. The series result is `10.0000-j0.4151 ohm`, with magnitude `10.0086 ohm`. The parallel result is `9.9998+j0.0415 ohm`, with magnitude `9.9999 ohm`.

For the parallel hand check, total admittance is approximately `0.1 - j0.000415 S`; its reciprocal is about `9.9998 + j0.0415 ohm`. The code's series result agrees with `std::complex<double>` at the displayed precision. Division by zero is rejected with a diagnostic. `std::complex<double>` would provide standard tested arithmetic with the same constant-time

operations and two-double state. One remaining limitation is that exact floating-point zero detection does not reject a near-zero denominator; real simulation software should define an appropriate tolerance and uncertainty policy.

Conclusion

The programs show how constructor delegation centralizes valid object creation, why owning raw pointers require coordinated copy and destruction behavior, how RAII links cleanup to object lifetime, and how overloaded operators can express useful domain operations. Integer paise avoids currency rounding; prefix increment avoids producing an unused old-value copy; and `std::string`/`std::complex` are safer standard alternatives for production code. The sanitizer runs confirm the deliberate shallow-copy double free and the clean deep-copy/Text/event paths, subject to the Windows LeakSanitizer limitation noted above.

Submission and authoring notes

The assignment PDF requests a GitHub repository named `DSA01-<RegisterNumber>-Assignment3`, while this work was added to the existing `DSA0101-192524005` repository specified in the request, on branch `assignment3-co3`. The naming mismatch remains unresolved because I cannot contact the instructor without their contact details and the user's authorization to send a message. The current name supplied by earlier assignment documentation is `Hitesh`; use the exact full name expected by the instructor if different. This report and code were prepared with AI assistance, and the student should review each section and be ready to explain it.

Final verification rerun

Before this rerun, assistant-prepared predictions were saved in `Task01/PREDICTION.md` through `Task07/PREDICTION.md`. The seven existing compiled executables were then run again; all eight runs (including both Q5 optimization levels) exited with code 0. Their output transcripts were refreshed in the task folders. This rerun checks the program behavior against the saved predictions; it does not replace the assignment's request for genuine screenshot files or establish an independent student prediction.

The package contains readable source/output PNG views for each task, along with the underlying source files and actual stdout transcripts. Those PNGs are renderings of the files, not screenshots of an IDE or live console. The Task 1 output was also shown in a genuine Notepad screen capture in the conversation, but that capture could not be saved into the repository. If grading requires direct screen captures, they remain outstanding.